

TOTAL SECTIONS	CURRICULUM PART	CORE MECHANICS	REFERENCE STANDARD
89 Modules + 3 Projects	Part 04 — Core Data Structures (2/3)	Indexing, Mutability, Iteration & FP	ECMAScript 2026 / V8 Runtime

Arrays — একসাথে অনেক Data সংরক্ষণ

JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ data structures-এর একটি হলো **Array**। এতদিন আমরা একটি variable-এ একটি value রাখতাম:

JAVASCRIPTSINGLE VALUE

```
let name = "Shariar";
```

কিন্তু ধরো ১০০ জন student-এর নাম রাখতে হবে। আলাদা ১০০টা variable বানানো practical নয়। তখন আমরা Array ব্যবহার করব:

JAVASCRIPTMULTIPLE VALUES ARRAY

```
let students = [
  "Shariar",
  "Rahim",
  "Karim",
  "Sakib"
];
```

PART 01 — ARRAY FUNDAMENTALS, CREATION, INDEXING & BASIC MUTATION (9.1 – 9.17)

9.1 Array কী?

Array হলো একটি ordered collection যেখানে একাধিক value একটি variable-এর মধ্যে রাখা যায়।

💡 REAL-LIFE EXAMPLE

একটি shopping cart:

```
Shopping Cart
├── Laptop
├── Mouse
├── Keyboard
└── Headphone
```

JavaScript:

9.1 JAVASCRIPTRUN

```
let cart = [
  "Laptop",
  "Mouse",
  "Keyboard",
  "Headphone"
];
```

9.2 Array তৈরি করা

সবচেয়ে common syntax:

9.2 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];
```

TEXT

```
["Apple", "Banana", "Mango"]
```

আরেকভাবে:

9.2 JAVASCRIPT

RUN

```
let fruits = new Array("Apple", "Banana", "Mango");
```

তবে modern JavaScript-এ সাধারণত:

9.2 JAVASCRIPT

RUN

```
[]
```

ব্যবহার করা হয়।

9.3 Array Index

Array-এর index শুরু হয়:

TEXT

```
0
```

থেকে।

9.3 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];
```

এটি:

Index:	0	1	2
	↓	↓	↓
Value:	Apple	Banana	Mango

9.4 Array থেকে Value Access

9.4 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
console.log(fruits[0]);
```

```
console.log(fruits[1]);
```

```
console.log(fruits[2]);
```

CONSOLE OUTPUT

```
Apple
```

```
Banana
```

```
Mango
```

9.5 প্রথম Element

9.5 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript"];
```

```
console.log(skills[0]);
```

TEXT

```
HTML
```

9.6 শেষ Element

9.6 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript"];

console.log(skills[skills.length - 1]);
```

TEXT

JavaScript

অথবা modern JavaScript:

9.6 JAVASCRIPT

RUN

```
console.log(skills.at(-1));
```

TEXT

JavaScript

9.7 Array Length

9.7 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

console.log(fruits.length);
```

TEXT

3

`length` হলো মোট element সংখ্যা।

9.8 Array-তে বিভিন্ন ধরনের Data রাখা যায়

JavaScript Array-তে একই Array-তে বিভিন্ন type রাখা সম্ভব।

9.8 JAVASCRIPT

RUN

```
let data = [
  "Shariar",
  23,
  true,
  null
];

console.log(data);
```

TEXT

["Shariar", 23, true, null]

এমনকি object/function-ও রাখা যায়।

9.8 JAVASCRIPT

RUN

```
let data = [
  "Ripon",
  23,
  { city: "Dhaka" },
  function () {
    console.log("Hello");
  }
];
```

তবে real-world application-এ একই ধরনের data রাখা সাধারণত বেশি পরিস্কার।

9.9 Array Element পরিবর্তন

Array-এর element directly পরিবর্তন করা যায়।

9.9 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits[1] = "Orange";

console.log(fruits);
```

CONSOLE OUTPUT

```
["Apple", "Orange", "Mango"]
```

এখানে:

Banana → Orange

হয়ে গেছে।

9.10 নতুন Element যোগ করা

শেষে element যোগ করতে:

9.10 JAVASCRIPT

RUN

```
push()
```

9.10 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana"];

fruits.push("Mango");

console.log(fruits);
```

TEXT

```
["Apple", "Banana", "Mango"]
```

9.11 `push()` একাধিক Element

9.11 JAVASCRIPT

RUN

```
let fruits = ["Apple"];

fruits.push("Banana", "Mango", "Orange");

console.log(fruits);
```

TEXT

```
["Apple", "Banana", "Mango", "Orange"]
```

9.12 `push()` কী Return করে?

এটা গুরুত্বপূর্ণ।

9.12 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana"];  
  
let result = fruits.push("Mango");  
  
console.log(result);
```

TEXT

3

`push()` নতুন Array length return করে।

9.13 শেষ Element Remove — `pop()`

9.13 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];  
  
let removed = fruits.pop();  
  
console.log(removed);  
console.log(fruits);
```

TEXT

Mango
["Apple", "Banana"]

9.14 `pop()` কী Return করে?

`pop()` যে element remove করে, সেটিই return করে।

9.14 JAVASCRIPT

RUN

```
let numbers = [10, 20, 30];  
  
let removed = numbers.pop();  
  
console.log(removed);
```

TEXT

30

9.15 প্রথমে Element যোগ — `unshift()`

Array-এর শুরুতে element যোগ করতে:

9.15 JAVASCRIPT

RUN

```
let fruits = ["Banana", "Mango"];  
  
fruits.unshift("Apple");  
  
console.log(fruits);
```

TEXT

["Apple", "Banana", "Mango"]

9.16 প্রথম Element Remove — `shift()`

9.16 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

let removed = fruits.shift();

console.log(removed);
console.log(fruits);
```

TEXT

Apple
["Banana", "Mango"]

9.17 Array-এর ৪টি Basic Method

এগুলো খুব ভালোভাবে মনে রাখবে:

Method	কাজ
<code>push()</code>	শেষে যোগ
<code>pop()</code>	শেষ থেকে remove
<code>unshift()</code>	শুরুতে যোগ
<code>shift()</code>	শুরু থেকে remove

Visual:

```
push()
  ↓
[ A, B, C ] → [ A, B, C, D ]

pop()
  ↓
[ A, B, C ] ← [ A, B, C, D ]

unshift()
  ↓
[ A, B, C ] → [ X, A, B, C ]

shift()
  ↓
[ A, B, C ] ← [ X, A, B, C ]
```

PART 02 — LOOPING, SEARCHING & SLICING VS SPLICING (9.18 – 9.35)

9.18 Array Loop — `for`

Array-এর প্রতিটি element নিয়ে কাজ করতে loop খুব গুরুত্বপূর্ণ।

9.18 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

CONSOLE OUTPUT

Apple
Banana
Mango

9.19

for ... of

Array-এর value সরাসরি পাওয়ার জন্য:

9.19 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

TEXT

```
Apple
Banana
Mango
```

Beginner-এর জন্য এটি খুব readable!

9.20

for ... in

Array-এর index পাওয়ার জন্য:

9.20 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let index in fruits) {
  console.log(index);
}
```

TEXT

```
0
1
2
```

Array-এর value দরকার হলে সাধারণত `for ... of` বেশি appropriate!

9.21

for ... of

vs

for ... in

Method

কী পাওয়া যায়

for ... of

Value

for ... in

Key/Index

9.21 JAVASCRIPT

RUN

```
for (let value of fruits) {
  console.log(value);
}
```

9.21 JAVASCRIPT

RUN

```
for (let index in fruits) {
  console.log(index);
}
```

9.22 includes()

Array-এর মধ্যে কোনো value আছে কিনা:

9.22 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript"];  
  
console.log(skills.includes("JavaScript"));
```

TEXT

true

না থাকলে:

9.22 JAVASCRIPT

RUN

```
console.log(skills.includes("Python"));
```

TEXT

false

9.23 indexOf()

কোন index-এ value আছে:

9.23 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript"];  
  
console.log(skills.indexOf("CSS"));
```

TEXT

1

না থাকলে:

9.23 JAVASCRIPT

RUN

```
console.log(skills.indexOf("Python"));
```

TEXT

-1

9.24 lastIndexOf()

একই value একাধিকবার থাকলে শেষ occurrence:

9.24 JAVASCRIPT

RUN

```
let numbers = [10, 20, 10, 30, 10];  
  
console.log(numbers.lastIndexOf(10));
```

TEXT

4

9.25 slice()

Array-এর একটি অংশ copy করতে:

9.25 JAVASCRIPT

RUN

```
let fruits = [
  "Apple",
  "Banana",
  "Mango",
  "Orange"
];

let result = fruits.slice(1, 3);

console.log(result);
```

TEXT

["Banana", "Mango"]

মনে রাখবে:

end index include হয় না।

9.26 slice() Original Array পরিবর্তন করেনা

9.26 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

let result = fruits.slice(0, 2);

console.log(result);
console.log(fruits);
```

TEXT

["Apple", "Banana"]
["Apple", "Banana", "Mango"]

9.27 Negative slice()

9.27 JAVASCRIPT

RUN

```
let fruits = [
  "Apple",
  "Banana",
  "Mango",
  "Orange"
];

console.log(fruits.slice(-2));
```

TEXT

["Mango", "Orange"]

9.28 splice() — খুব গুরুত্বপূর্ণ 🔥

splice() Array-এর মধ্যে:

- add
- remove
- replace

সবই করতে পারে।

Syntax:

9.28 JAVASCRIPT

RUN

```
array.splice(start, deleteCount, item1, item2, ...);
```

9.29 splice() দিয়ে Remove

9.29 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.splice(1, 1);

console.log(fruits);
```

TEXT

```
["Apple", "Mango"]
```

এখানে:

TEXT

```
start = 1
deleteCount = 1
```

তাই `Banana` remove হয়েছে।

9.30 splice() দিয়ে Add

9.30 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Mango"];

fruits.splice(1, 0, "Banana");

console.log(fruits);
```

TEXT

```
["Apple", "Banana", "Mango"]
```

এখানে:

```
1 → কোন index থেকে
0 → কিছু remove করব না
"Banana" → নতুন value
```

9.31 splice() দিয়ে Replace

9.31 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.splice(1, 1, "Orange");

console.log(fruits);
```

TEXT

```
["Apple", "Orange", "Mango"]
```

9.32 splice() কী Return করে?

যে elements remove হয়েছে, সেগুলো একটি Array হিসেবে return করে।

9.32 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

let removed = fruits.splice(1, 1);

console.log(removed);
```

TEXT

["Banana"]

9.33 slice() vs splice()

এটা Interview-এ খুব common।

slice()	splice()
Copy/Extract	Add/Remove/Replace
Original Array পরিবর্তন করে না	Original Array পরিবর্তন করে
নতুন Array return করে	Removed elements-এর Array return করে
slice(start,end)	splice(start,deleteCount, ...)

সহজে মনে রাখো:

slice = কেটে copy নেওয়া

splice = মূল Array-তে operation করা

9.34 join()

Array → String করতে:

9.34 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript"];

console.log(skills.join(", "));
```

TEXT

HTML, CSS, JavaScript

9.35 join() Real-life Example

9.35 JAVASCRIPT

RUN

```
let tags = ["web", "javascript", "frontend"];

let result = tags.join(" #");

console.log("#" + result);
```

TEXT

#web #javascript #frontend

9.36 `reverse()`

Array-এর order উল্টে দেয়।

9.36 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4];

numbers.reverse();

console.log(numbers);
```

TEXT

```
[4, 3, 2, 1]
```

⚠ `reverse()` original Array পরিবর্তন করে।

9.37 `sort()`

Array sort করতে:

9.37 JAVASCRIPT

RUN

```
let fruits = ["Mango", "Apple", "Banana"];

fruits.sort();

console.log(fruits);
```

TEXT

```
["Apple", "Banana", "Mango"]
```

9.38 গুরুত্বপূর্ণ — Number Sort

এই জায়গায় beginner-রা অনেক ভুল করে।

9.38 JAVASCRIPT

RUN

```
let numbers = [10, 2, 30, 5];

numbers.sort();

console.log(numbers);
```

Output হতে পারে:

TEXT

```
[10, 2, 30, 5]
```

কারণ default `sort()` values-কে String হিসেবে compare করে।

9.39 Numeric Sort — Ascending

9.39 JAVASCRIPT

RUN

```
let numbers = [10, 2, 30, 5];

numbers.sort((a, b) => a - b);

console.log(numbers);
```

TEXT

```
[2, 5, 10, 30]
```

9.40 Numeric Sort — Descending

9.40 JAVASCRIPT

RUN

```
let numbers = [10, 2, 30, 5];

numbers.sort((a, b) => b - a);

console.log(numbers);
```

TEXT

[30, 10, 5, 2]

9.41 forEach()

Array-এর প্রতিটি element-এর উপর function চালাতে:

9.41 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.forEach(function (fruit) {
  console.log(fruit);
});
```

TEXT

Apple
Banana
Mango

9.42 forEach() with Arrow Function

Modern style:

9.42 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.forEach(fruit => {
  console.log(fruit);
});
```

9.43 forEach() Index

9.43 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.forEach((fruit, index) => {
  console.log(index, fruit);
});
```

TEXT

0 Apple
1 Banana
2 Mango

9.44

map()



map() একটি Array-এর প্রতিটি element transform করে **নতুন Array** তৈরি করে।

9.44 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4];

let doubled = numbers.map(number => number * 2);

console.log(doubled);
```

TEXT

```
[2, 4, 6, 8]
```

9.45

map()

Real-life Example

Product price:

9.45 JAVASCRIPT

RUN

```
let prices = [100, 200, 300];

let discountedPrices = prices.map(price => price * 0.9);

console.log(discountedPrices);
```

TEXT

```
[90, 180, 270]
```

9.46

forEach()

vs

map()

forEach()	map()
প্রতিটি element-এর জন্য কাজ করে	প্রতিটি element transform করে
নতুন Array return করে না	নতুন Array return করে
Side effects-এর জন্য useful	Data transformation-এর জন্য useful

9.47

filter()



কোনো condition অনুযায়ী elements বেছে নিতে:

9.47 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5, 6];

let evenNumbers = numbers.filter(number => number % 2 === 0);

console.log(evenNumbers);
```

TEXT

```
[2, 4, 6]
```

9.48 filter() Real-life Example

9.48 JAVASCRIPT

RUN

```
let products = [
  { name: "Laptop", price: 70000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 }
];

let expensive = products.filter(product => product.price > 5000);

console.log(expensive);
```

TEXT

Laptop

বাস্তবে object নিয়ে কাজ করলে পুরো object-ই থাকবে filtered Array-তে।

PART 04 — ADVANCED SEARCH & AGGREGATION — FIND, SOME, EVERY, REDUCE (9.49 – 9.57)

9.49 find()

Condition অনুযায়ী **প্রথম matching element** বের করতে:

9.49 JAVASCRIPT

RUN

```
let numbers = [10, 20, 30, 40];

let result = numbers.find(number => number > 20);

console.log(result);
```

TEXT

30

কারণ প্রথম 20-এর পর 30 condition satisfy করেছে।

9.50 find() না পেলো

9.50 JAVASCRIPT

RUN

```
let numbers = [10, 20, 30];

let result = numbers.find(number => number > 100);

console.log(result);
```

TEXT

undefined

9.51 findIndex()

Matching element-এর index:

9.51 JAVASCRIPT

RUN

```
let numbers = [10, 20, 30, 40];

let index = numbers.findIndex(number => number > 20);

console.log(index);
```

TEXT

2

9.52

some()

কমপক্ষে একটি element condition satisfy করে কিনা:

9.52 JAVASCRIPT

RUN

```
let numbers = [1, 3, 5, 8];  
  
console.log(numbers.some(number => number % 2 === 0));
```

TEXT

true

কারণ 8 even।

9.53

every()

সবগুলো element condition satisfy করে কিনা:

9.53 JAVASCRIPT

RUN

```
let numbers = [2, 4, 6, 8];  
  
console.log(numbers.every(number => number % 2 === 0));
```

TEXT

true

কিন্তু:

9.53 JAVASCRIPT

RUN

```
let numbers = [2, 4, 5, 8];  
  
console.log(numbers.every(number => number % 2 === 0));
```

TEXT

false

9.54

reduce()



reduce() Array-এর অনেকগুলো value থেকে একটি final result তৈরি করতে ব্যবহৃত হয়।

💡 EXAMPLE — SUM

9.54 JAVASCRIPT

RUN

```
let numbers = [10, 20, 30, 40];  
  
let total = numbers.reduce((sum, number) => {  
  return sum + number;  
}, 0);  
  
console.log(total);
```

TEXT

100

9.55 `reduce()` কীভাবে কাজ করে?

TEXT

```
Initial = 0  
  
0 + 10 = 10  
10 + 20 = 30  
30 + 30 = 60  
60 + 40 = 100
```

শেষে:

TEXT

100

9.56 `reduce()` Real-life — Shopping Cart

9.56 JAVASCRIPT

RUN

```
let prices = [1000, 2000, 3000];  
  
let total = prices.reduce((sum, price) => {  
  return sum + price;  
}, 0);  
  
console.log(total);
```

TEXT

6000

এটি shopping cart-এর total price calculation-এর basic idea।

9.57 `reduce()` দিয়ে Maximum

9.57 JAVASCRIPT

RUN

```
let numbers = [10, 50, 20, 80, 30];  
  
let max = numbers.reduce((highest, number) => {  
  return number > highest ? number : highest;  
}, numbers[0]);  
  
console.log(max);
```

TEXT

80

9.58 Array Destructuring

Array থেকে সরাসরি variable তৈরি করা যায়।

9.58 JAVASCRIPT

RUN

```
let colors = ["Red", "Green", "Blue"];

let [first, second, third] = colors;

console.log(first);
console.log(second);
console.log(third);
```

TEXT

```
Red
Green
Blue
```

9.59 Destructuring Skip

9.59 JAVASCRIPT

RUN

```
let colors = ["Red", "Green", "Blue"];

let [first, , third] = colors;

console.log(first);
console.log(third);
```

TEXT

```
Red
Blue
```

9.60 Destructuring Default Value

9.60 JAVASCRIPT

RUN

```
let colors = ["Red"];

let [first, second = "Blue"] = colors;

console.log(first);
console.log(second);
```

TEXT

```
Red
Blue
```

9.61 Rest with Array Destructuring

9.61 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5];

let [first, second, ...remaining] = numbers;

console.log(first);
console.log(second);
console.log(remaining);
```

TEXT

```
1
2
[3, 4, 5]
```

9.62 Spread Operator with Array

Array copy করতে:

9.62 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana"];

let newFruits = [...fruits];

console.log(newFruits);
```

TEXT

```
["Apple", "Banana"]
```

9.63 Array Combine

9.63 JAVASCRIPT

RUN

```
let frontend = ["HTML", "CSS"];
let backend = ["Node.js", "Express"];

let skills = [...frontend, ...backend];

console.log(skills);
```

TEXT

```
["HTML", "CSS", "Node.js", "Express"]
```

9.64 Array Copy — Important

9.64 JAVASCRIPT

RUN

```
let original = [1, 2, 3];

let copy = [...original];

copy.push(4);

console.log(original);
console.log(copy);
```

TEXT

```
[1, 2, 3]
[1, 2, 3, 4]
```

9.65 Nested Array

একটি Array-এর ভিতরে আরেকটি Array থাকতে পারে।

9.65 JAVASCRIPT

RUN

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(matrix);
```

9.66 Nested Array Access

9.66 JAVASCRIPT

RUN

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6]
];

console.log(matrix[0][1]);
```

TEXT

2

কারণ:

```
matrix[0] → [1, 2, 3]
```

```
matrix[0][1] → 2
```

9.67 2D Array Real-life Example

একটি classroom-এর marks:

9.67 JAVASCRIPT

RUN

```
let marks = [
  [80, 75, 90],
  [70, 85, 88],
  [92, 89, 95]
];
```

এখানে প্রতিটি inner Array একজন student-এর marks হতে পারে।

9.68 Array-এর মধ্যে Object

Real-world JavaScript application-এ এটি খুব common:

9.68 JAVASCRIPT

RUN

```
let students = [
  {
    name: "Shariar",
    age: 23
  },
  {
    name: "Rahim",
    age: 22
  }
];

console.log(students[0].name);
```

TEXT

Shariar

এই concept React/Node.js-এ অত্যন্ত গুরুত্বপূর্ণ।

9.69 Array.isArray()

কোনো value Array কিনা check করতে:

9.69 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS"];

console.log(Array.isArray(skills));
```

TEXT

true

আর:

9.69 JAVASCRIPT

RUN

```
console.log(Array.isArray("Hello"));
```

TEXT

false

9.70 Array বনাম Object

Array	Object
Ordered collection	Key-value data
Index ব্যবহার করে	Key ব্যবহার করে
<code>users[0]</code>	<code>user.name</code>
List-এর জন্য useful	Entity/data model-এর জন্য useful

Example:

9.70 JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JS"];
```

Object:

9.70 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23
};
```

9.71 Empty Array

9.71 JAVASCRIPT

RUN

```
let items = [];  
  
console.log(items.length);
```

TEXT

0

তারপর:

9.71 JAVASCRIPT

RUN

```
items.push("Laptop");  
  
console.log(items);
```

TEXT

["Laptop"]

9.72 Sparse Array

9.72 JAVASCRIPT

RUN

```
let numbers = [];  
  
numbers[3] = 100;  
  
console.log(numbers);
```

এতে মাঝখানে empty slots তৈরি হতে পারে।

TEXT

[empty, empty, empty, 100]

এভাবে Array তৈরি করা সাধারণত avoid করা ভালো।

9.73 Array Reference

এটা খুব গুরুত্বপূর্ণ।

9.73 JAVASCRIPT

RUN

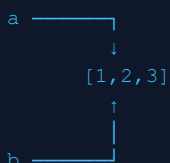
```
let a = [1, 2, 3];  
  
let b = a;  
  
b.push(4);  
  
console.log(a);
```

TEXT

[1, 2, 3, 4]

কেন?

কারণ:



a এবং **b** একই Array object-কে reference করছে।

9.74 Array Copy করার সঠিক উপায়

9.74 JAVASCRIPT

RUN

```
let a = [1, 2, 3];  
  
let b = [...a];  
  
b.push(4);  
  
console.log(a);  
console.log(b);
```

TEXT

```
[1, 2, 3]  
[1, 2, 3, 4]
```

PART 06 — ADVANCED METHODS, CONSTRUCTORS & ECOSYSTEM REFERENCE (9.75 – 9.89)

9.75 concat()

দুটি Array combine:

9.75 JAVASCRIPT

RUN

```
let a = [1, 2];  
let b = [3, 4];  
  
let result = a.concat(b);  
  
console.log(result);
```

TEXT

```
[1, 2, 3, 4]
```

Spread দিয়েও:

9.75 JAVASCRIPT

RUN

```
let result = [...a, ...b];
```

9.76 flat()

Nested Array flatten করতে:

9.76 JAVASCRIPT

RUN

```
let numbers = [1, 2, [3, 4], [5, 6]];  
  
console.log(numbers.flat());
```

TEXT

```
[1, 2, 3, 4, 5, 6]
```

9.77 Deep Nested flat()

9.77 JAVASCRIPT

RUN

```
let numbers = [
  1,
  [2, [3, [4]]]
];

console.log(numbers.flat(Infinity));
```

TEXT

[1, 2, 3, 4]

9.78 Array.from()

Array-like বা iterable data থেকে Array তৈরি করতে:

9.78 JAVASCRIPT

RUN

```
let text = "Hello";

let chars = Array.from(text);

console.log(chars);
```

TEXT

["H", "e", "l", "l", "o"]

9.79 Array.of()

Values দিয়ে Array তৈরি:

9.79 JAVASCRIPT

RUN

```
let numbers = Array.of(10, 20, 30);

console.log(numbers);
```

TEXT

[10, 20, 30]

9.80 Array.from() দিয়ে Generate

9.80 JAVASCRIPT

RUN

```
let numbers = Array.from(
  { length: 5 },
  (_, index) => index + 1
);

console.log(numbers);
```

TEXT

[1, 2, 3, 4, 5]

9.81 Important: `fill()`

Array-এর নির্দিষ্ট জায়গায় একই value বসাতে:

9.81 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5];  
  
numbers.fill(0);  
  
console.log(numbers);
```

TEXT

```
[0, 0, 0, 0, 0]
```

9.82 `fill()` Partial

9.82 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5];  
  
numbers.fill(0, 1, 3);  
  
console.log(numbers);
```

TEXT

```
[1, 0, 0, 4, 5]
```

9.83 `copyWithin()`

Array-এর ভিতরের কিছু element অন্য জায়গায় copy করতে:

9.83 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5];  
  
numbers.copyWithin(0, 3);  
  
console.log(numbers);
```

TEXT

```
[4, 5, 3, 4, 5]
```

এটি তুলনামূলকভাবে কম ব্যবহৃত method, কিন্তু JavaScript Array API-এর অংশ হিসেবে জানা ভালো।

9.84 `flatMap()`

`map()` + `flat()` একসাথে করার মতো।

9.84 JAVASCRIPT

RUN

```
let numbers = [1, 2, 3];  
  
let result = numbers.flatMap(number => [number, number * 2]);  
  
console.log(result);
```

TEXT

```
[1, 2, 2, 4, 3, 6]
```

9.85

entries()

Index এবং value দুটো একসাথে:

9.85 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let [index, fruit] of fruits.entries()) {
  console.log(index, fruit);
}
```

TEXT

```
0 Apple
1 Banana
2 Mango
```

9.86

keys()

Index পাওয়া যায়:

9.86 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let key of fruits.keys()) {
  console.log(key);
}
```

TEXT

```
0
1
2
```

9.87

values()

Values:

9.87 JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];

for (let value of fruits.values()) {
  console.log(value);
}
```

TEXT

```
Apple
Banana
Mango
```

9.88 Array Method Categories

Add / Remove

TEXT

```
push()  
pop()  
shift()  
unshift()  
splice()
```

Search

TEXT

```
includes()  
indexOf()  
lastIndexOf()  
find()  
findIndex()
```

Transform

TEXT

```
map()  
flatMap()
```

Filter / Check

TEXT

```
filter()  
some()  
every()
```

Calculation

TEXT

```
reduce()
```

Copy / Extract

TEXT

```
slice()  
concat()
```

Ordering

TEXT

```
sort()  
reverse()
```

Convert

TEXT

```
join()  
flat()
```

9.89 Array Methods – সবচেয়ে গুরুত্বপূর্ণ Table

Method	কাজ	Original Array পরিবর্তন?
<code>push()</code>	শেষে যোগ	✓
<code>pop()</code>	শেষে remove	✓
<code>shift()</code>	শুরু থেকে remove	✓
<code>unshift()</code>	শুরুতে যোগ	✓
<code>splice()</code>	add/remove/replace	✓
<code>slice()</code>	অংশ copy	✗
<code>map()</code>	transform	✗
<code>filter()</code>	filter	✗
<code>find()</code>	প্রথম match	✗
<code>findIndex()</code>	প্রথম match-এর index	✗
<code>some()</code>	অন্তত একটি match	✗
<code>every()</code>	সব match	✗
<code>reduce()</code>	single result	✗
<code>includes()</code>	value আছে কিনা	✗
<code>indexOf()</code>	index খোঁজা	✗
<code>join()</code>	Array → String	✗
<code>concat()</code>	Array combine	✗
<code>reverse()</code>	reverse	✓
<code>sort()</code>	sort	✓
<code>fill()</code>	value fill	✓
<code>flat()</code>	nested flatten	✗

PROJECT 1 🔥 Real-life Mini Project — Shopping Cart

এখন Array-এর অনেক concept একসাথে ব্যবহার করি।

JAVASCRIPT

RUN

```
let cart = [
  {
    name: "Laptop",
    price: 70000
  },
  {
    name: "Mouse",
    price: 1000
  },
  {
    name: "Keyboard",
    price: 3000
  }
];

console.log("Products:");

cart.forEach(product => {
  console.log(product.name);
});

let total = cart.reduce((sum, product) => {
  return sum + product.price;
}, 0);

console.log("Total:", total);
```

CONSOLE OUTPUT

```
Products:
Laptop
Mouse
Keyboard

Total: 74000
```

এখানে আমরা ব্যবহার করেছি:

TEXT

```
Array
Object
forEach()
reduce()
```

এই pattern বাস্তব e-commerce application-এ খুব common।

PROJECT 2 🔥 Real-life Mini Project — Student Result

JAVASCRIPT

RUN

```
let marks = [80, 75, 90, 65, 88];

let total = marks.reduce((sum, mark) => sum + mark, 0);

let average = total / marks.length;

let passed = marks.filter(mark => mark >= 40);

console.log("Total:", total);
console.log("Average:", average);
console.log("Passed:", passed);
```

CONSOLE OUTPUT

```
Total: 398
Average: 79.6
Passed: [80, 75, 90, 65, 88]
```

PROJECT 3 🔥 Real-life Mini Project — Search Products

JAVASCRIPT

RUN

```
let products = [
  "HP Laptop",
  "Dell Laptop",
  "Logitech Mouse",
  "Mechanical Keyboard"
];

let search = "laptop";

let result = products.filter(product =>
  product.toLowerCase().includes(search.toLowerCase())
);

console.log(result);
```

CONSOLE OUTPUT

```
[
  "HP Laptop",
  "Dell Laptop"
]
```

এটি basic product search-এর ধারণা।

তোমাকে এগুলো অবশ্যই ভালোভাবে আয়ত্ত করতে হবে:

TEXT

1. Array কী
2. Array Index
3. length
4. Accessing elements
5. Updating elements
6. push()
7. pop()
8. shift()
9. unshift()
10. for loop
11. for ... of
12. forEach()
13. includes()
14. indexOf()
15. slice()
16. splice()
17. join()
18. reverse()
19. sort()
20. map()
21. filter()
22. find()
23. findIndex()
24. some()
25. every()
26. reduce()
27. Array Destructuring
28. Spread Operator
29. Nested Arrays
30. Array of Objects
31. Array Reference
32. Array Copy
33. concat()
34. flat()
35. flatMap()
36. Array.from()
37. Array.of()
38. fill()
39. entries()
40. keys()
41. values()
42. Array.isArray()

Practice 1 — Basic Array

JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];
```

এগুলো বের করো:

TEXT

```
Apple  
Mango  
3
```

Practice 2 — Add & Remove

JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS"];
```

JavaScript এবং React যোগ করো।

তারপর শেষের element remove করো।

Practice 3 — Search

JAVASCRIPT

RUN

```
let skills = ["HTML", "CSS", "JavaScript", "React"];
```

Check করো:

```
JavaScript আছে → true  
Python আছে → false
```

Practice 4 — Filter

JAVASCRIPT

RUN

```
let numbers = [10, 15, 20, 25, 30, 35];
```

শুধু even numbers বের করো।

Expected:

TEXT

```
[10, 20, 30]
```

Practice 5 — Map

JAVASCRIPT

RUN

```
let numbers = [1, 2, 3, 4, 5];
```

প্রতিটি number-এর double বের করো।

Expected:

TEXT

```
[2, 4, 6, 8, 10]
```

Practice 6 — Reduce

JAVASCRIPT

RUN

```
let prices = [100, 200, 300, 400];
```

Total বের করো।

Expected:

TEXT

1000

Practice 7 — Find

JAVASCRIPT

RUN

```
let numbers = [10, 25, 40, 55, 70];
```

50 -এর বেশি প্রথম number বের করো।

Expected:

TEXT

55

Practice 8 — Sort

JAVASCRIPT

RUN

```
let numbers = [50, 10, 80, 20, 30];
```

Ascending:

TEXT

[10, 20, 30, 50, 80]

Descending:

TEXT

[80, 50, 30, 20, 10]

Practice 9 — Destructuring

JAVASCRIPT

RUN

```
let student = ["Shariar", 23, "CSE"];
```

Destructuring করে:

TEXT

```
name
age
department
```

অলাদা variable বানাও।

Practice 10 — Mini Project

JAVASCRIPT

RUN

```
let cart = [
  { name: "Laptop", price: 70000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 },
  { name: "Monitor", price: 15000 }
];
```

এমন program তৈরি করো যা:

- সব product-এর নাম দেখাবে
- সব product-এর total price বের করবে
- টাকার বেশি product বের করবে
- সবচেয়ে বেশি দামের product বের করবে
- product-এর নামগুলো একটি নতুন Array-তে রাখবে

SUMMARY

 Chapter 9 Final Summary — এক নজরে Array

এক কথায়:

Array = এক variable-এর মধ্যে ordered multiple values রাখার ব্যবস্থা।

সবচেয়ে বেশি মনে রাখবে:

JAVASCRIPT

RUN

```
let fruits = ["Apple", "Banana", "Mango"];
```

তারপর:

JAVASCRIPT

RUN

```
fruits.push("Orange");    // Add
fruits.pop();             // Remove last
fruits.unshift("Orange"); // Add first
fruits.shift();           // Remove first
```

Search:

JAVASCRIPT

RUN

```
fruits.includes("Apple");
fruits.indexOf("Apple");
```

Extract:

JAVASCRIPT

RUN

```
fruits.slice(0, 2);
```

Modify:

JAVASCRIPT

RUN

```
fruits.splice(1, 1);
```

Transform:

JAVASCRIPT

RUN

```
fruits.map( ... );
```

Filter:

JAVASCRIPT

RUN

```
fruits.filter( ... );
```

Find:

JAVASCRIPT

RUN

```
fruits.find( ... );
```

Calculate:

JAVASCRIPT

RUN

```
numbers.reduce( ... );
```

Loop:

JAVASCRIPT

RUN

```
fruits.forEach( ... );
```

Combine:

JAVASCRIPT

RUN

```
[ ... array1, ... array2];
```

Destructure:

JAVASCRIPT

RUN

```
let [a, b, c] = array;
```

যদি এখনই সব method মুখস্থ না হয়, অন্তত এগুলো শক্ত করো:

TEXT

1. push()
2. pop()
3. slice()
4. splice()
5. forEach()
6. map()
7. filter()
8. reduce()

এরপর **Chapter 10 — Objects**-এ আমরা JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ data structure-এর আরেকটি অংশ শিখব: **Object, Properties, Methods, Nested Objects, Object Access, Add/Update/Delete, `this`, Object Destructuring, Spread, `Object.keys()`, `Object.values()`, `Object.entries()`, Optional Chaining, Object References, Shallow Copy, এবং Array of Objects**।